

In Java, `wait()`, `sleep()`, and `join()` all make a thread **stop executing temporarily**, but they serve very different purposes.

The easiest way to remember them is:

Method	Purpose	Releases lock?	Defined in
<code>wait()</code>	Wait for another thread to notify	✓ Yes	<code>Object</code>
<code>sleep()</code>	Pause for a specific time	✗ No	<code>Thread</code>
<code>join()</code>	Wait for another thread to finish	✗ No*	<code>Thread</code>

* `join()` internally uses `wait()`, but from the caller's perspective, the caller does not voluntarily release some arbitrary lock it holds.

1. `sleep()` — Pause the current thread

`sleep()` tells the **currently executing thread** to pause for a specified amount of time.

Example

Java

```
public class SleepExample {  
  
    public static void main(String[] args) throws InterruptedException {  
  
        System.out.println("Task started");  
  
        // Current thread sleeps for 2 seconds  
        Thread.sleep(2000);  
  
        System.out.println("Task completed");  
    }  
}
```

Output:

```
Task started  
  
// waits 2 seconds
```

Task completed

Important point

`sleep()` does not release a lock.

Consider:

Java

```
public synchronized void method() throws InterruptedException {  
  
    System.out.println("Lock acquired");  
  
    Thread.sleep(5000);  
  
    System.out.println("Lock released");  
}
```

While the thread is sleeping:

```
Thread  
  ↓  
Has lock  
  ↓  
sleep(5000)  
  ↓  
Still has lock  
  ↓  
5 seconds  
  ↓  
Continues  
  ↓  
Releases lock
```

Another thread trying to enter the synchronized method may have to wait.

When do we use `sleep()` ?

Typical use cases:

- Retry delays
- Polling
- Simulating processing time
- Scheduled delays
- Rate limiting in simple scenarios

Example:

Java

```
for (int i = 1; i <= 5; i++) {  
  
    System.out.println("Processing: " + i);  
  
    Thread.sleep(1000);  
}
```

2. `join()` — Wait for another thread to finish

`join()` is used when one thread wants to wait until another thread completes.

Consider:

Java

```
Thread t1 = new Thread(() -> {  
    System.out.println("Worker thread running");  
});  
  
t1.start();  
  
t1.join();  
  
System.out.println("Main thread continues");
```

The execution is:

```
Main Thread  
  |  
  | start()  
  ↓  
Worker Thread  
  |  
  | doing work  
  |  
  ↓  
finishes  
  |  
  ↓  
Main Thread continues
```

Without `join()` :

```
Main Thread  
  |
```

```
| start()
↓
Worker Thread
|
↓
Main Thread continues immediately
```

Complete example

Java

```
public class JoinExample {

    public static void main(String[] args)
        throws InterruptedException {

        Thread worker = new Thread(() -> {

            System.out.println("Worker started");

            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            System.out.println("Worker finished");
        });

        worker.start();

        // Main thread waits until worker finishes
        worker.join();

        System.out.println("Main thread continues");
    }
}
```

Output:

```
Worker started
// 3 seconds
Worker finished
Main thread continues
```

Why use `join()`?

Suppose you have:

```
Thread A → Generate report
Thread B → Main thread
```

You don't want Thread B to continue until the report is generated.

Java

```
reportThread.start();

reportThread.join();

System.out.println("Report is ready");
```

3. `wait()` — Thread communication

`wait()` is different from `sleep()` and `join()`.

It is primarily used for **communication/synchronization between threads**.

`wait()` belongs to:

Java

```
java.lang.Object
```

not `Thread`.

You normally use it with:

Java

```
synchronized
```

Basic example

Java

```
public class WaitExample {

    public static void main(String[] args)
        throws InterruptedException {

        Object lock = new Object();

        Thread thread = new Thread(() -> {
```

```

        synchronized (lock) {

            try {
                System.out.println("Thread is waiting...");

                lock.wait();

                System.out.println("Thread resumed");

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    });

    thread.start();

    Thread.sleep(2000);

    synchronized (lock) {

        System.out.println("Main thread sends notification");

        lock.notify();
    }
}
}

```

Output:

```

Thread is waiting...

// 2 seconds

Main thread sends notification

Thread resumed

```

4. What happens internally with `wait()`?

This is the **most important interview point**.

Suppose:

Java

```
synchronized (lock) {  
  
    lock.wait();  
  
}
```

The thread:

1. Must own the object's monitor.
2. Calls `wait()`.
3. Releases the monitor/lock.
4. Enters the waiting state.
5. Another thread calls `notify()` or `notifyAll()`.
6. The waiting thread becomes eligible to continue.
7. It must reacquire the same monitor before returning from `wait()`.

Conceptually:

```
Thread A  
|  
| synchronized(lock)  
↓  
Acquires lock  
|  
| wait()  
↓  
Releases lock  
|  
↓  
WAITING  
|  
| notify()  
↓  
Runnable / eligible  
|  
↓  
Reacquires lock  
|  
↓  
Continues
```

This is fundamentally different from `sleep()`.

5. `sleep()` vs `wait()`

This is one of the most common Java interview questions.

`sleep()`

Java

```
synchronized (lock) {  
  
    Thread.sleep(5000);  
  
}
```

The thread:

```
Has lock  
  ↓  
sleep()  
  ↓  
Still has lock
```

`wait()`

Java

```
synchronized (lock) {  
  
    lock.wait();  
  
}
```

The thread:

```
Has lock  
  ↓  
wait()  
  ↓  
Releases lock  
  ↓  
Waits
```

Therefore:

`sleep()` pauses execution but does not release the monitor. `wait()` releases the monitor and waits for notification.

6. `wait()` must be called inside synchronized context

This is important:

Java

```
lock.wait();
```

should normally be called while holding `lock`'s monitor:

Java

```
synchronized (lock) {  
  
    lock.wait();  
  
}
```

If you do:

Java

```
lock.wait(); // ❌
```

without owning the monitor, Java throws:

```
IllegalMonitorStateException
```

7. `notify()` and `notifyAll()`

`wait()` is normally used together with:

Java

```
notify()
```

or:

Java

```
notifyAll()
```

`notify()`

Wakes **one** waiting thread.

Java

```
synchronized (lock) {  
    lock.notify();  
}
```

```
notifyAll()
```

Wakes **all** threads waiting on that monitor.

Java

```
synchronized (lock) {  
    lock.notifyAll();  
}
```

But waking doesn't mean the thread immediately executes. It must first **reacquire the lock**.

8. Real-world example: Producer and Consumer

Imagine:

```
Producer  
  ↓  
Adds data  
  ↓  
Consumer  
  ↓  
Processes data
```

If there is no data, the consumer should wait.

Java

```
public class ProducerConsumer {  
  
    private static final Object lock = new Object();  
  
    private static boolean dataAvailable = false;  
  
    public static void main(String[] args) {  
  
        Thread consumer = new Thread(() -> {  
  
            synchronized (lock) {  
  
                while (!dataAvailable) {  
  
                    try {  
                        System.out.println("Consumer waiting...");
```

```

        lock.wait();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

    System.out.println("Consumer processing data");
}

});

Thread producer = new Thread() -> {

    synchronized (lock) {

        System.out.println("Producer produced data");

        dataAvailable = true;

        lock.notify();
    }
});

consumer.start();

try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

producer.start();
}
}

```

Conceptually:

```

Consumer
  ↓
No data
  ↓
wait()
  ↓
Releases lock
  ↓
WAITING

```

Then:

```
Producer
↓
Produces data
↓
dataAvailable = true
↓
notify()
↓
Consumer wakes
↓
Reacquires lock
↓
Processes data
```

Why use `while` instead of `if`?

Prefer:

Java

```
while (!dataAvailable) {
    lock.wait();
}
```

instead of:

Java

```
if (!dataAvailable) {
    lock.wait();
}
```

because a waiting thread can experience **spurious wakeups**, and because the condition should always be rechecked after waking.

9. The easiest way to remember all three

Think about three different situations.

`sleep()`

"I need a break for 5 seconds."

Java

```
Thread.sleep(5000);
```

Pause for time

`join()`

"I need to wait until that thread finishes."

Java

```
worker.join();
```

Wait for another thread to finish

`wait()`

"I can't continue until another thread tells me that something has happened."

Java

```
lock.wait();
```

Wait for a condition/notification

10. Interview comparison

Feature	<code>wait()</code>	<code>sleep()</code>	<code>join()</code>
Class	Object	Thread	Thread
Purpose	Thread communication	Time-based pause	Wait for thread completion
Releases monitor?	✓ Yes	✗ No	✗ Not the caller's arbitrary monitor
Requires synchronized context?	✓ Yes	✗ No	✗ No

Feature	<code>wait()</code>	<code>sleep()</code>	<code>join()</code>
Can be interrupted?	✓ Yes	✓ Yes	✓ Yes
Needs <code>notify()</code> ?	Usually ✓	✗	✗
Typical state	<code>WAITING</code>	<code>TIMED_WAITING</code>	<code>WAITING</code> / <code>TIMED_WAITING</code>
Common use	Producer-consumer	Delay/retry	Thread dependency

★ Interview answer

If the interviewer asks:

"What's the difference between `wait()`, `sleep()`, and `join()`?"

A strong answer is:

`sleep()` is used to pause the current thread for a specified duration and it does not release any monitor lock held by the thread. `wait()` is used for inter-thread communication; it must be called while owning the object's monitor, and it releases that monitor while waiting for `notify()` or `notifyAll()`. `join()` is used when one thread needs to wait for another thread to complete. So, `sleep()` is time-based waiting, `wait()` is condition/notification-based waiting, and `join()` is thread-completion-based waiting.